

第 23 章

人脸识别

人脸识别是指程序对输入的人脸图像进行判断，并识别出其对应的人的过程。人脸识别程序像我们人类一样，“看到”一张人脸后就能够分辨出这个人是家人、朋友还是明星。

当然，要实现人脸识别，首先要判断当前图像内是否出现了人脸，即人脸检测。只有检测到图像中出现了人脸，才能根据人脸判断这个人到底是谁。

本章分别介绍人脸检测和人脸识别的基本原理，并分别给出了使用 OpenCV 实现它们的简单案例。

23.1 人脸检测

当我们预测的是离散值时，进行的是“分类”。例如，预测一个孩子能否成为一名优秀的运动员，其实就是看他是被划分为“好苗子”还是“普通孩子”的分类。对于只涉及两个类别的“二分类”任务，我们通常将其中一个类称为“正类”（正样本），另一个类称为“负类”（反类、负样本）。

例如，在人脸检测中，主要任务是构造能够区分包含人脸实例和不包含人脸实例的分类器。这些实例被称为“正类”（包含人脸图像）和“负类”（不包含人脸图像）。

本节介绍分类器的基本构造方法，以及如何调用 OpenCV 中训练好的分类器实现人脸检测。

23.1.1 基本原理

OpenCV 提供了三种不同的训练好的级联分类器，下面简单介绍其中涉及的一些概念。

1. 级联分类器

通常情况下，分类器需要对多个图像特征进行识别。例如，识别一个动物到底是狗（正类）还是其他动物（负类），我们可能需要根据多个条件进行判断，这样比较下来是非常烦琐的。但是，如果首先就比较它们有几条腿：

- 有“四条腿”的动物被判断为“可能为狗”，并对此范围内的对象继续进行分析和判断。
- 没有“四条腿”的动物直接被否决，即不可能为狗。

这样，仅仅比较腿的数目，根据这个特征就能排除样本集中大量的负类（例如鸡、鸭、鹅等不是狗的其他动物实例）。

级联分类器就是基于这种思路，将多个简单的分类器按照一定的顺序级联而成的。



级联分类器的基本原理如图 23-1 所示。

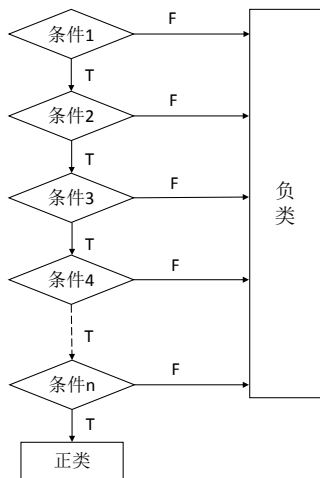


图 23-1 级联分类器示意图

级联分类器的优势是，在开始阶段仅进行非常简单的判断，就能够排除明显不符合要求的实例。在开始阶段被排除的负类，不再参与后续分类，这样能极大地提高后面分类的速度。这有点像我们经常收到的骗子短信，大多数人通常一眼就能识别出这些短信是骗人的，也不可能上当受骗。骗子们随机大量发送大多数人明显不会上当受骗的短信，这种做法虽然看起来非常蠢，但总还是会有人上当。这些短信，在最开始的阶段经过简单的筛选过滤就能够将完全不可能上当的人排除在外。不回复短信的人，是不可能上当的；而回复短信的人，才是目标人群。这样，骗子轻易地就识别并找到了目标人群，能够更专注地“服务”于他们的“最终目标人群”（不断地进行短信互动），从而有效地避免了与“非目标人群”（不回复短信的人群）发生进一步的接触而“浪费”时间和精力。

OpenCV 提供了用于训练级联分类器的工具，也提供了训练好的用于人脸定位的级联分类器，都可以作为现成的资源使用。

2. Haar级联分类器

OpenCV 提供了已经训练好的 Haar 级联分类器用于人脸定位。Haar 级联分类器的实现，经过了以下漫长的历史：

- 首先，有学者提出了使用 Haar 特征用于人脸检测，但是此时 Haar 特征的运算量超级大，这个方案并不实用。
- 接下来，有学者提出了简化 Haar 特征的方法，让使用 Haar 特征检测人脸的运算变得简单易行，同时提出了使用级联分类器提高分类效率。
- 后来，又有学者提出用于改进 Haar 的类 Haar 方案，为人脸定义了更多特征，进一步提高了人脸检测的效率。

下面用一个简单的例子来叙述上述方案。假设有两幅 4×4 大小的图像，如图 23-2 所示。针对这两幅图像，我们可以通过简单的计算来判断它们在左右关系这个维度是否具有相关性。

128	108	47	27
96	76	88	68

图 23-2 图像示例

用两幅图像左侧像素值之和减去右侧像素值之和：

- 针对左图， $\text{sum}(\text{左侧像素}) - \text{sum}(\text{右侧像素}) = (128+96) - (108+76) = 40$
- 针对右图， $\text{sum}(\text{左侧像素}) - \text{sum}(\text{右侧像素}) = (47+88) - (27+68) = 40$

这两幅图像中，“左侧像素值之和”减去“右侧像素值之和”都是 40。所以，可以认为在“左侧像素值之和”减去“右侧像素值之和”这个角度，这两幅图像具有一定的相关性。

进一步扩展，我们可以从更多的角度考虑图像的特征。学者 Papageorgiou 等人提出了如图 23-3 所示的 Haar 特征，这些特征包含垂直特征、水平特征和对角特征。他们利用这些特征分别实现了行人检测（*Pedestrian Detection Using Wavelet Templates*）和人脸检测（*A General Framework For Object Detection*）。

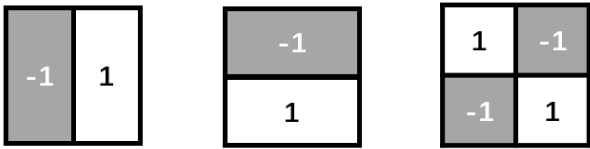


图 23-3 Haar 特征

Haar 特征反映的是图像的灰度变化，它将像素划分为模块后求差值。Haar 特征用黑白两种矩形框组合成特征模板，在特征模板内，用白色矩形像素块的像素和减去黑色矩形像素块的像素和来表示该模板的特征。经过上述处理后，人脸部的一些特征就可以使用矩形框的差值简单地表示了。比如，眼睛的颜色比脸颊的颜色要深，鼻梁两侧的颜色比鼻梁的颜色深，唇部的颜色比唇部周围的颜色深。

关于 Harr 特征中的矩形框，有如下 3 个变量。

- 矩形位置：矩形框要逐像素地划过（遍历）整个图像获取每个位置的差值。
- 矩形大小：矩形的大小可以根据需要做任意调整。
- 矩形类型：包含垂直、水平、对角等不同类型。

上述 3 个变量保证了能够细致全面地获取图像的特征信息。但是，变量的个数越多，特征的数量也会越多。例如，仅一个 24×24 大小的检测窗口内的特征数量就接近 20 万个。由于计算量过大，该方案并不实用，除非有人提出能够简化特征的方案。

后来，Viola 和 Jones 两位学者在论文 *Rapid Object Detection Using A Boosted Cascade Of Simple Features* 和 *Robust Real-time Face Detection* 中提出了使用积分图快速计算 Haar 特征的方法。他们提出通过构造“积分图（Integral Image）”，让 Haar 特征能够通过查表法和有限次简单运算快速获取，极大地减少了运算量。同时，在这两篇文章中，他们提出了通过构造级联分

类器让不符合条件的背景图像（负样本）被快速地抛弃，从而能够将算力运用在可能包含人脸的对象上。

为了进一步提高效率，Lienhart 和 Maydt 两位学者，在论文 *An Extended Set Of Haar-Like Features For Rapid Object Detection* 中提出对 Haar 特征库进行扩展。他们将 Haar 特征进一步划分为如图 23-4 所示的 4 类：

- 4 个边特征。
- 8 个线特征。
- 2 个中心点特征。
- 1 个对角特征。

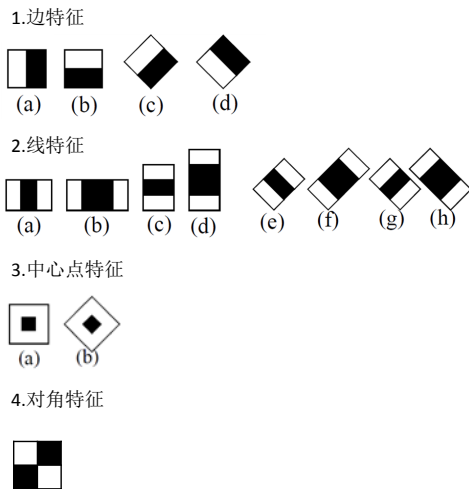


图 23-4 Haar 扩展特征

Lienhart 和 Maydt 两位学者认为在实际使用中，对角特征（见图 23-4 底部）和线特征中的“e”和“g”（见图 23-4 的第 2 行，(e)图和(g)图）是相近的，因此通常情况下无须重复计算。同时，该论文还给出了计算 Haar 特征数的方法、快速计算方法，以及级联分类器的构造方法等内容。

OpenCV 在上述研究的基础上，实现了将 Haar 级联分类器用于人脸部的特征的定位。我们可以直接调用 OpenCV 自带的 Haar 级联特征分类器来实现人脸定位。

除此以外，OpenCV 还提供了使用 Hog 特征和 LBP 算法的级联分类器。Hog 级联分类器主要用于行人检测，这里不再赘述。有关 LBP 算法的内容请参考 23.2 节。

23.1.2 级联分类器的使用

为了训练针对特定类型对象的级联分类器，OpenCV 提供了专门的软件工具。在 OpenCV 根目录下的 build 文件夹下，查找 build\x86\vc12\bin 目录（不同的 OpenCV 版本，路径会略有差异），会找到 opencv_createsamples.exe 和 opencv_traincascade.exe，这两个.exe 文件可以用来训练级联分类器。

训练级联分类器很耗时，如果训练的数据量较大，可能需要好几天才能完成。在 OpenCV



中，有一些训练好的级联分类器供用户使用。这些分类器可以用来检测人脸、脸部特征（眼睛、鼻子）、人类和其他物体。这些级联分类器以 XML 文件的形式存放在 OpenCV 源文件的 data 目录下，加载不同级联分类器的 XML 文件就可以实现对不同对象的检测。

OpenCV 自带的级联分类器存储在 OpenCV 根文件夹的 data 文件夹下。该文件夹包含三个子文件夹：haarcascades、hogcascades、lbpcascades，里面分别存储的是 Harr 级联分类器、HOG 级联分类器、LBP 级联分类器。

其中，Harr 级联分类器多达 20 多种（随着版本更新还会继续增加），提供了对多种对象的检测功能。部分级联分类器如表 23-1 所示。

表 23-1 级联分类器

XML 文件名	级联分类器类型
haarcascade_eye.xml	眼睛检测
haarcascade_eye_tree_eyeglasses.xml	眼镜检测
haarcascade_mcs_nose.xml	鼻子检测
haarcascade_mcs_mouth.xml	嘴巴检测
haarcascade_smile.xml	表情检测
hogcascade_pedestrians.xml	行人检测
lbpcascade_frontalface.xml	正面人脸检测
lbpcascade_profileface.xml	人脸检测
lbpcascade_silverware.xml	金属检测

加载级联分类器的语法格式为：

```
<CascadeClassifier object> = cv2.CascadeClassifier( filename )
```

式中，filename 是分类器的路径和名称。

下面的代码是一个调用实例：

```
faceCascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
```

使用级联分类器时需要注意：如果你是通过在 anaconda 中使用 pip 的方式安装的 OpenCV，则无法直接获取级联分类器的 XML 文件。可以通过以下两种方式获取需要的级联分类器 XML 文件：

- 安装 OpenCV 后，在其安装目录下的 data 文件夹内查找 XML 文件。
- 直接在网络上找到相应 XML 文件，下载并使用。

同样，如果使用 opencv_createsamples.exe 和 opencv_traincascade.exe，也需要采用上述方式获取 XML 文件。

23.1.3 函数介绍

在 OpenCV 中，人脸检测使用的是 cv2.CascadeClassifier.detectMultiScale() 函数，它可以检测出图片中所有的人脸。该函数由分类器对象调用，其语法格式为：

```
objects = cv2.CascadeClassifier.detectMultiScale( image[, scaleFactor[,  
minNeighbors[, flags[, minSize[, maxSize]]]] ] )
```

式中各个参数及返回值的含义为：

- **image**: 待检测图像，通常为灰度图像。
- **scaleFactor**: 表示在前后两次相继的扫描中，搜索窗口的缩放比例。
- **minNeighbors**: 表示构成检测目标的相邻矩形的最小个数。默认情况下，该值为 3，意味着有 3 个以上的检测标记存在时，才认为人脸存在。如果希望提高检测的准确率，可以将该值设置得更大，但同时可能会让一些人脸无法被检测到。
- **flags**: 该参数通常被省略。在使用低版本 OpenCV（OpenCV 1.X 版本）时，它可能会被设置为 `CV_HAAR_DO_CANNY_PRUNING`，表示使用 Canny 边缘检测器来拒绝一些区域。
- **minSize**: 目标的最小尺寸，小于这个尺寸的目标将被忽略。
- **maxSize**: 目标的最大尺寸，大于这个尺寸的目标将被忽略。
- **objects**: 返回值，目标对象的矩形框向量组。

23.1.4 案例介绍

本节通过一个实例来说明如何实现人脸检测。

【例 23.1】 使用函数 `cv2.CascadeClassifier.detectMultiScale()` 检测一幅图像内的人脸。

根据题目的要求，编写代码如下：

```
import cv2
# 读取待检测的图像
image = cv2.imread('dface3.jpg')
# 获取 XML 文件，加载人脸检测器
faceCascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
# 色彩转换，转换为灰度图像
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# 调用函数 detectMultiScale
faces = faceCascade.detectMultiScale(
    gray,
    scaleFactor = 1.15,
    minNeighbors = 5,
    minSize = (5,5)
)
print(faces)
# 打印输出的测试结果
print("发现{0}个人脸!".format(len(faces)))
# 逐个标注人脸
for(x,y,w,h) in faces:
    # cv2.rectangle(image, (x,y), (x+w,y+w), (0,255,0),2) #矩形标注
    cv2.circle(image, (int((x+x+w)/2), int((y+y+h)/2)), int(w/2), (0,255,0),2)
# 显示结果
cv2.imshow("dect", image)
# 保存检测结果
cv2.imwrite("re.jpg", image)
cv2.waitKey(0)
```



```
cv2.destroyAllWindows()
```

运行上述程序，会显示如图 23-5 所示的图像。程序将图像内的 5 个人脸使用 5 个圆圈标注出来。

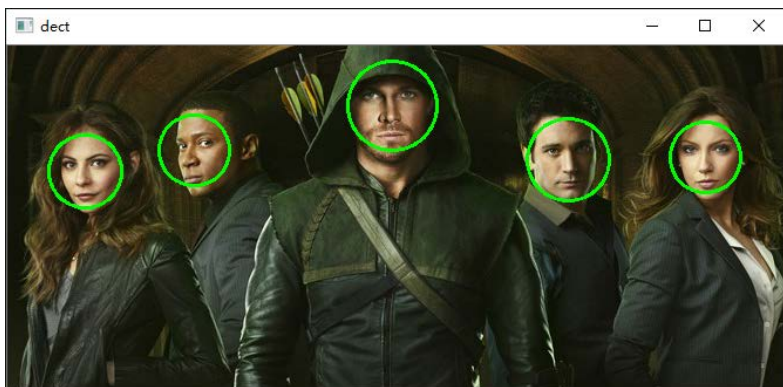


图 23-5 人脸检测结果

同时，在控制台会显示检测到的人脸的具体位置信息及个数，具体结果如下：

```
[[129 59 61 61]
 [ 35 76 62 62]
 [565 65 61 61]
 [443 62 71 71]
 [290 13 77 77]]
```

发现 5 个人脸！

23.2 LBPH 人脸识别

人脸识别的第一步，就是要找到一个模型可以用简洁又具有差异性的方式准确反映出每个人脸的特征。识别人脸时，先将当前人脸采用与前述同样的方式提取特征，再从已有特征集中找出当前特征的最邻近样本，从而得到当前人脸的标签。

OpenCV 提供了三种人脸识别方法，分别是 LBPH 方法、EigenFishfaces 方法、Fisherfaces 方法。本节主要对 LBPH 方法进行简单介绍。

LBPH (Local Binary Patterns Histogram, 局部二值模式直方图) 所使用的模型基于 LBP (Local Binary Pattern, 局部二值模式) 算法。LBP 最早是被作为一种有效的纹理描述算子提出的，由于在表述图像局部纹理特征上效果出众而得到广泛应用。

23.2.1 基本原理

LBP 算法的基本原理是，将像素点 A 的值与其最邻近的 8 个像素点的值逐一比较：

- 如果 A 的像素值大于其临近点的像素值，则得到 0。
- 如果 A 的像素值小于其临近点的像素值，则得到 1。

最后，将像素点 A 与其周围 8 个像素点比较所得到的 0、1 值连起来，得到一个 8 位的二

进制序列，将该二进制序列转换为十进制数作为点 A 的 LBP 值。

下面以图 23-6 中左侧 3×3 区域的中心点（像素值为 76 的点）为例，说明如何计算该点的 LBP 值。计算时，以其像素值 76 作为阈值，对其 8 邻域像素进行二值化处理，

- 将像素值大于 76 的像素点处理为 1。例如，其邻域中像素值为 128、251、99、213 的点，都被处理为 1，填入对应的像素点位置上。
- 将像素值小于 76 的像素点处理为 0。例如，其邻域中像素值为 36、9、11、48 的点，都被处理为 0，填入对应的像素点位置上。

根据上述计算，可以得到图 23-6 中右图所示的二值结果。

128	36	251
48	76	9
11	213	99

1	0	1
0		0
0	1	1

图 23-6 LBP 原理示意图

完成二值化以后，任意指定一个开始位置，将得到的二值结果进行序列化，组成一个 8 位的二进制数。例如，从当前像素点的正上方开始，以顺时针为序得到二进制序列“01011001”。

最后，将二进制序列“01011001”转换为所对应的十进制数“89”，作为当前中心点的像素值，如图 23-7 所示。

128	36	251
48	76	9
11	213	99

	89	

图 23-7 中心点的处理结果

对图像逐像素用以上方式进行处理，就得到 LBP 特征图像，这个特征图像的直方图被称为 LBPH，或称为 LBP 直方图。

为了得到不同尺度下的纹理结构，还可以使用圆形邻域，将计算扩大到任意大小的邻域内。圆形邻域可以用(P, R)表示，其中 P 表示圆形邻域内参与运算的像素点个数， R 表示邻域的半径。

例如，在图 23-8 中就分别采用了不同的圆形邻域。

- 左侧使用的是(4, 1)邻域，比较当前像素与邻域内 4 个像素点的像素值大小，使用的半径是 1。
- 右侧使用的是(8, 2)邻域，比较当前像素与邻域内 8 个像素点的像素值大小，使用的半径是 2。在参与比较的 8 个邻域像素点中，部分邻域可能不会直接取实际存在的某个位置上的像素点，而是通过计算构造一个“虚拟”像素值来与当前像素点进行比较。

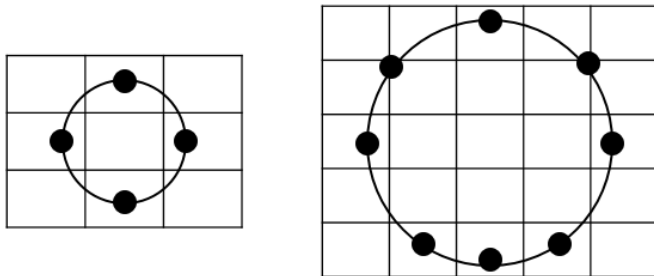


图 23-8 圆形邻域示意图

人脸的整体灰度由于受到光线的影响，经常会发生变化，但是人脸各部分之间的相对灰度会基本保持一致。LBP 的主要思想是以当前点与其邻域像素的相对关系作为处理结果，正是因为这一点，在图像灰度整体发生变化（单调变化）时，从 LBP 算法中提取的特征能保持不变。因此，LBP 在人脸识别中得到了广泛的应用。

从上面的介绍可以看到，LBP 特征与 Haar 特征很相似，都是图像的灰度变化特征。

23.2.2 函数介绍

在 OpenCV 中，可以用函数 `cv2.face.LBPHFaceRecognizer_create()` 生成 LBPH 识别器实例模型，然后应用 `cv2.face.FaceRecognizer.train()` 函数完成训练，最后用 `cv2.face.FaceRecognizer.predict()` 函数完成人脸识别。

下面分别介绍上述三个函数。

1. 函数 `cv2.face.LBPHFaceRecognizer_create()`

函数 `cv2.face.LBPHFaceRecognizer_create()` 的语法格式为：

```
retval = cv2.face.LBPHFaceRecognizer_create( [ , radius[, neighbors[,  
grid_x[, grid_y[, threshold]]]]])
```

其中全部的参数都是可选的，含义如下：

- **radius**: 半径值，默认值为 1。
- **neighbors**: 邻域点的个数，默认采用 8 邻域，根据需要可以计算更多的邻域点。
- **grid_x**: 将 LBP 特征图像划分为一个个单元格时，每个单元格在水平方向上的像素个数。该参数值默认为 8，即将 LBP 特征图像在行方向上以 8 个像素为单位分组。
- **grid_y**: 将 LBP 特征图像划分为一个个单元格时，每个单元格在垂直方向上的像素个数。该参数值默认为 8，即将 LBP 特征图像在列方向上以 8 个像素为单位分组。
- **threshold**: 在预测时所使用的阈值。如果大于该阈值，就认为没有识别到任何目标对象。

2. 函数 `cv2.face.FaceRecognizer.train()`

函数 `cv2.face.FaceRecognizer.train()` 对每个参考图像计算 LBPH，得到一个向量。每个人脸都是整个向量集中的一个点。该函数的语法格式为：

```
None = cv2.face.FaceRecognizer.train( src, labels )
```

式中各个参数的含义为:

- **src**: 训练图像, 用来学习的人脸图像。
- **labels**: 标签, 人脸图像所对应的标签。

该函数没有返回值。

3. 函数 `cv2.face_FaceRecognizer.predict()`

函数 `cv2.face_FaceRecognizer.predict()` 对一个待测人脸图像进行判断, 寻找与当前图像距离最近的人脸图像。与哪个人脸图像最近, 就将当前待测图像标注为其对应的标签。当然, 如果待测图像与所有人脸图像的距离都大于函数 `cv2.face.LBPHFaceRecognizer_create()` 中参数 `threshold` 所指定的距离值, 则认为没有找到对应的结果, 即无法识别当前人脸。

函数 `cv2.face_FaceRecognizer.predict()` 的语法格式为:

```
label, confidence = cv2.face_FaceRecognizer.predict( src )
```

式中参数与返回值的含义为:

- **src**: 需要识别的人脸图像。
- **label**: 返回的识别结果标签。
- **confidence**: 返回的置信度评分。置信度评分用来衡量识别结果与原有模型之间的距离。0 表示完全匹配。通常情况下, 认为小于 50 的值是可以接受的, 如果该值大于 80 则认为差别较大。

23.2.3 案例介绍

本节使用 OpenCV 的 LBPH 模块实现一个简单的人脸识别示例。

【例 23.2】 完成一个简单的人脸识别程序。

本例中有两个人, 每个人有两幅人脸图像, 用于机器学习。然后, 我们用程序识别第 5 幅人脸图像 (为其中一个人的人脸), 观察识别结果。

用于学习的 4 幅人脸图像如图 23-9 所示, 从左到右图像的名称分别为 `a1.png`、`a2.png`、`b1.png`、`b2.png`。



图 23-9 【例 23.2】用于学习的人脸图像

这 4 幅图像中, 前两幅图像是同一个人, 将其标签设定为 “0”; 后两幅图像是同一个人, 将其标签设定为 “1”。

用于识别的人脸图像如图 23-10 所示, 该图像的名称为 `a3.png`。





图 23-10 【例 23.2】待识别的人脸图像

根据题目的要求，编写代码如下：

```
import cv2
import numpy as np
images=[]
images.append(cv2.imread("a1.png",cv2.IMREAD_GRAYSCALE))
images.append(cv2.imread("a2.png",cv2.IMREAD_GRAYSCALE))
images.append(cv2.imread("b1.png",cv2.IMREAD_GRAYSCALE))
images.append(cv2.imread("b2.png",cv2.IMREAD_GRAYSCALE))
labels=[0,0,1,1]
#print(labels)
recognizer = cv2.face.LBPHFaceRecognizer_create()
recognizer.train(images, np.array(labels))
predict_image=cv2.imread("a3.png",cv2.IMREAD_GRAYSCALE)
label,confidence= recognizer.predict(predict_image)
print("label=",label)
print("confidence=",confidence)
```

运行上述程序，识别结果为：

```
label= 0
confidence= 67.6856704732354
```

从输出结果可以看到，标签值为“0”，置信区间值为 68。这说明图像 a3.png 被识别为标签 0 所对应的人脸图像，即认为当前待识别图像 a3.png 中的人与图像 a1.png、a2.png 中的是同一个人。

23.3 EigenFaces 人脸识别

EigenFaces 通常也被称为特征脸，它使用主成分分析 (Principal Component Analysis, PCA) 方法将高维的人脸数据处理为低维数据后（降维），再进行数据分析和处理，获取识别结果。

23.3.1 基本原理

在现实世界中，很多信息的表示是有冗余的。例如，表 23-2 所列出一组圆的参数中就存在冗余信息。

表 23-2 一组圆的参数

序号	半径	直径	周长	面积
1	3	6	19	28
2	1	2	6	3
3	2	4	13	13
4	7	14	44	154
5	1	2	6	3
6	5	10	31	79
7	1	2	6	3
8	6	12	38	113

在表 23-2 所示的参数中，各个参数之间存在着非常强的相关性：

- 直径 = 2*半径
- 周长 = 2* π *半径
- 面积 = π *半径*半径

可以看到，直径、周长和面积都可以通过半径计算得到。

在进行数据分析时，如果我们希望更直观地看到这些参数的值，就需要获取所有字段的值。但是，在比较圆的面积大小时，仅使用半径就足够了，此时其他信息对于我们来说就是“冗余”的。

因此，我们可以理解“半径”就是表 23-2 所列数据中的“主成分”，我们将“半径”从上述数据中提取出来供后续分析使用，就实现了“降维”。

当然，上面例子的数据非常简单、易于理解，而在大多数情况下，我们要处理的数据是比较复杂的。很多时候，我们可能无法直接判断哪些数据是关键的“主成分”，所以就要通过 PCA 方法将复杂数据内的“主成分”分析出来。

EigenFaces 就是对原始数据使用 PCA 方法进行降维，获取其中的主成分信息，从而实现人脸识别的方法。

23.3.2 函数介绍

OpenCV 通过函数 `cv2.face.EigenFaceRecognizer_create()`生成特征脸识别器实例模型，然后应用 `cv2.face_FaceRecognizer.train()`函数完成训练，最后用 `cv2.face_FaceRecognizer.predict()`函数完成人脸识别。

1. 函数`cv2.face.EigenFaceRecognizer_create()`

函数 `cv2.face.EigenFaceRecognizer_create()`的语法格式为：

```
retval = cv2.face.EigenFaceRecognizer_create( [, num_components[, threshold]] )
```

式中的两个参数都是可选参数，含义如下：

- `num_components`：在 PCA 中要保留的分量个数。当然，该参数值通常要根据输入数据来具体确定，并没有一定之规。一般来说，80 个分量就足够了。
- `threshold`：进行人脸识别时所采用的阈值。

2. 函数 `cv2.face_FaceRecognizer.train()`

函数 `cv2.face_FaceRecognizer.train()` 对每个参考图像进行 EigenFaces 计算, 得到一个向量。每个人脸都是整个向量集中的一个点。该函数的语法格式为:

```
None = cv2.face_FaceRecognizer.train( src, labels )
```

式中各个参数的含义为:

- **src**: 训练图像, 用来学习的人脸图像。
- **labels**: 人脸图像所对应的标签。

该函数没有返回值。

3. 函数 `cv2.face_FaceRecognizer.predict()`

函数 `cv2.face_FaceRecognizer.predict()` 在对一个待测人脸图像进行判断时, 会寻找与当前图像距离最近的人脸图像。与哪个人脸图像最接近, 就将待测图像识别为其对应的标签。该函数的语法格式为:

```
label, confidence = cv2.face_FaceRecognizer.predict( src )
```

式中各个参数及返回值的含义为:

- **src**: 需要识别的人脸图像。
- **label**: 返回的识别结果标签。
- **confidence**: 返回的置信度评分。置信度评分用来衡量识别结果与原有模型之间的距离。0 表示完全匹配。该参数值通常在 0 到 20 000 之间, 只要低于 5000, 都被认为是相当可靠的识别结果。注意, 这个范围与 LBPH 的置信度评分值的范围是不同的。

23.3.3 案例介绍

本节使用 OpenCV 的 EigenFaces 模块实现一个简单的人脸识别示例。

【例 23.3】 使用 EigenFaces 模块完成一个简单的人脸识别程序。

本例中用于学习的 4 幅人脸图像如图 23-11 所示, 从左到右图像的名称分别为 e01.png、e02.png、e11.png、e12.png。



图 23-11 【例 23.3】用于学习的人脸图像

这 4 幅图像中, 前两幅图像是同一个人, 将其标签设定为“0”; 后两幅图像是同一个人, 将其标签设定为“1”。

待识别的人脸图像如图 23-12 所示, 该图像的名称为 eTest.png。



图 23-12 【例 23.3】待识别的人脸图像

根据题目的要求，编写代码如下：

```
import cv2
import numpy as np
images=[]
images.append(cv2.imread("e01.png",cv2.IMREAD_GRAYSCALE))
images.append(cv2.imread("e02.png",cv2.IMREAD_GRAYSCALE))
images.append(cv2.imread("e11.png",cv2.IMREAD_GRAYSCALE))
images.append(cv2.imread("e12.png",cv2.IMREAD_GRAYSCALE))
labels=[0,0,1,1]
#print(labels)
recognizer = cv2.face.EigenFaceRecognizer_create()
recognizer.train(images, np.array(labels))
predict_image=cv2.imread("eTest.png",cv2.IMREAD_GRAYSCALE)
label,confidence= recognizer.predict(predict_image)
print("label=",label)
print("confidence=",confidence)
```

运行上述程序，识别结果为：

```
label= 0
confidence= 1592.4971123726827
```

从输出结果可以看到，eTest.png 被识别为标签“0”所对应的人脸图像，即认为图像 eTest.png 与图像 e01.png、e02.png 中的是同一个人。

23.4 Fisherfaces 人脸识别

PCA 方法是 EigenFaces 方法的核心，它找到了最大化数据总方差特征的线性组合。不可否认，EigenFaces 是一种非常有效的方法，但是它的缺点在于在操作过程中会损失许多特征信息。因此，在一些情况下，如果损失的信息正好是用于分类的关键信息，必然会导致无法完成分类。

Fisherfaces 采用 LDA（Linear Discriminant Analysis，线性判别分析）实现人脸识别。线性判别识别最早由 Fisher 在 1936 年提出，是一种经典的线性学习方法，也被称为“Fisher 判别分析法”。

23.4.1 基本原理

线性判别分析在对特征降维的同时考虑类别信息。其思路是：在低维表示下，相同的类应



该紧密地聚集在一起；不同的类别应该尽可能地分散开，并且它们之间的距离尽可能地远。简单地说，线性判别分析就是要尽力满足以下两个要求：

- 类别间的差别尽可能地大。
- 类别内的差别尽可能地小。

做线性判别分析时，首先将训练样本集投影到一条直线 A 上，让投影后的点满足：

- 同类间的点尽可能地靠近。
- 异类间的点尽可能地远离。

做完投影后，将待测样本投影到直线 A 上，根据投影点的位置判定样本的类别，就完成了识别。

例如，图 23-13 所示的是一组训练样本集。现在需要找到一条直线，让所有的训练样本满足：同类间的距离最近，异类间的距离最远。

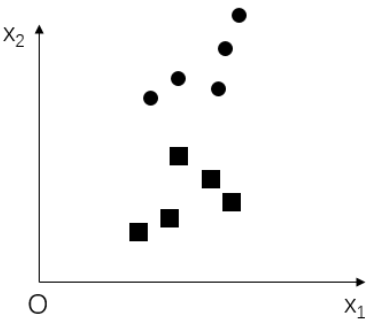


图 23-13 训练样本集

图 23-14 的左图和右图中分别有两条不同的投影线 L1 和 L2，将图 23-13 中的样本分别投影到这两条线上，可以看到样本集在 L2 上的投影效果要好于在 L1 上的投影效果。

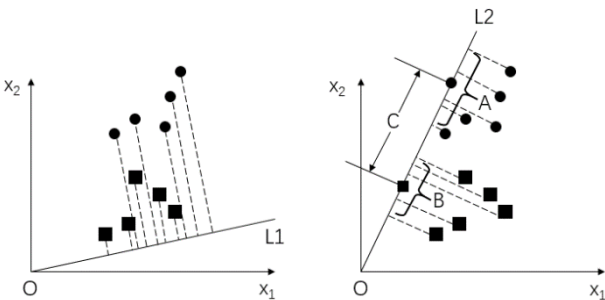


图 23-14 投影示例

线性判别分析就是要找到一条最优的投影线。以图 23-14 中右图投影为例，要满足：

- A、B 组内的点之间尽可能地靠近
- C 的两个端点之间的距离（类间距离）尽可能地远

找到一条这样的直线后，如果要判断某个待测样本的分组，可以直接将该样本点向投影线投影，然后根据投影点的位置来判断其所属类别。

例如,在图 23-15 中,三角形样本点 U 向投影线投影后,其投影点落在圆点的投影范围内,则认为待测样本点 U 属于圆点所在的分类。

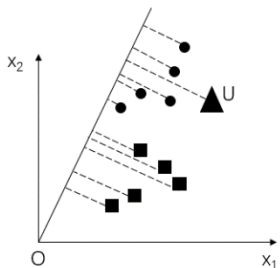


图 23-15 判断样本点的分类

23.4.2 函数介绍

在 OpenCV 中,通过函数 `cv2.face.FisherFaceRecognizer_create()` 生成 Fisherfaces 识别器实例模型,然后应用 `cv2.face.FaceRecognizer.train()` 函数完成训练,用 `cv2.face.FaceRecognizer.predict()` 函数完成人脸识别。

1. 函数 `cv2.face.FisherFaceRecognizer_create()`

函数 `cv2.face.FisherFaceRecognizer_create()` 的语法格式为:

```
retval = cv2.face.FisherFaceRecognizer_create( [ ,
                                                num_components[, threshold]] )
```

式中的两个参数都是可选参数,它们的含义为:

- **num_components**: 使用 Fisherfaces 准则进行线性判别分析时保留的成分数量。可以采用默认值“0”,让函数自动设置合适的成分数量。
- **threshold**: 进行识别时所用的阈值。如果最近的距离比设定的阈值 `threshold` 还要大,函数会返回“-1”。

2. 函数 `cv2.face.FaceRecognizer.train()`

函数 `cv2.face.FaceRecognizer.train()` 对每个参考图像进行 Fisherfaces 计算,得到一个向量。每个人脸都是整个向量集中的一个点。该函数的语法格式为:

```
None = cv2.face.FaceRecognizer.train( src, labels )
```

式中各个参数的含义为:

- **src**: 训练图像,即用来学习的人脸图像。
- **labels**: 人脸图像所对应的标签。

该函数没有返回值。

3. 函数 `cv2.face.FaceRecognizer.predict()`

函数 `cv2.face.FaceRecognizer.predict()` 在对一个待测人脸图像进行判断时,寻找与其距离最近的人脸图像。与哪个人脸图像最接近,就将待测图像识别为其对应的标签。该函数的语法格



式为：

```
label, confidence = cv2.face_FaceRecognizer.predict( src )
```

式中的参数与返回值的含义为：

- **src**：需要识别的人脸图像。
- **label**：返回的识别结果的标签。
- **confidence**：置信度评分。置信度评分用来衡量识别结果与原有模型之间的距离。0 表示完全匹配。该值通常在 0 到 20 000 之间，若低于 5000，就认为是相当可靠的识别结果。需要注意，该评分值的范围与 EigenFaces 方法的评分值范围一致，与 LBPH 方法的评分值范围不一致。

23.4.3 案例介绍

本节使用 OpenCV 的 FisherFaces 模块实现一个简单的人脸识别示例。

【例 23.4】 使用 FisherFaces 完成一个简单的人脸识别程序。

本例中用于学习的 4 幅人脸图像如图 23-16 所示，它们的名称从左至右分别为 f01.png、f02.png、f11.png、f12.png。



图 23-16 【例 23.4】用于训练的人脸图像

这 4 幅图像中，前两幅图像是同一个人，将其标签设定为“0”；后两幅图像是同一个人，将其标签设定为“1”。

待识别的人脸图像如图 23-17 所示，该图像的名称为 fTest.png。



图 23-17 待识别人脸图像

根据题目的要求，编写代码如下：

```
import cv2
import numpy as np
images=[]
images.append(cv2.imread("f01.png",cv2.IMREAD_GRAYSCALE))
```

```

images.append(cv2.imread("f02.png", cv2.IMREAD_GRAYSCALE))
images.append(cv2.imread("f11.png", cv2.IMREAD_GRAYSCALE))
images.append(cv2.imread("f12.png", cv2.IMREAD_GRAYSCALE))
labels=[0,0,1,1]
#print(labels)
recognizer = cv2.face.FisherFaceRecognizer_create()
recognizer.train(images, np.array(labels))
predict_image=cv2.imread("fTest.png", cv2.IMREAD_GRAYSCALE)
label,confidence= recognizer.predict(predict_image)
print("label=", label)
print("confidence=", confidence)

```

运行上述程序，识别结果为：

```

label= 0
confidence= 92.5647623298737

```

从输出结果可以看到，fTest.png 被识别为标签“0”所对应的人脸图像，即认为人脸图像 fTest.png 与图像 f01.png、f02.png 所表示的是同一个人。

23.5 人脸数据库

下面我们对几个常用的资源进行简单说明。

1. CAS-PEAL

CAS-PEAL (Chinese Academy of Sciences - Pose, Expression, Accessory, and Lighting) 是中科院计算技术研究所 2003 年完成的包含 1040 位志愿者 (其中有 595 位男性和 445 位女性) 的共 99 594 幅人脸图像的数据库。数据集的所有图像都是在专门的采集环境中采集的，涵盖了姿态、表情、饰物和光照 4 种主要变化条件，部分人脸图像具有背景、距离和时间跨度的变化。对每个人，在水平半圆形架子上设置了 9 部相机同时捕获其不同姿态的图像，还用了上下两个镜头 (也即共 18 幅图像)。此外，这个数据库还考虑了 5 种表情、6 种附件 (3 副眼镜、3 顶帽子) 和 15 个照明方向。目前，该数据库面向研究开放了其子集“CAS-PEAL-R1”，子集内包含 1040 人的 30 900 幅图像。

2. AT&T Facedatabase

即以前的 ORL 人脸数据集(The ORL Database of Faces)。ORL 是“Olivetti & Oracle Research Lab”的简称，该实验室后来被 AT&T 收购，更名为“AT&T Laboratories Cambridge”，2002 年 AT&T 宣布结束对该实验室的资助。目前该数据集由剑桥大学计算机实验室 (Cambridge University Computer Laboratory) 的数据技术组 (The Digital Technology Group) 负责维护。这个人脸数据集包含了 1992 年到 1994 年在实验室内拍摄的一些人脸图像。

该数据集包含 40 个人的 400 幅图像。这些图像具有不同的拍摄时间、不同的光线、不同的面部表情 (睁开/闭眼、微笑/不微笑) 和不同的面部细节 (戴眼镜/不戴眼镜)。所有的图像都是在亮度均匀的背景下拍摄的，被拍摄对象处于直立状态，拍摄正脸 (部分图像具有较小幅度的侧脸)。



数据集内所有的图像是以 PGM 格式存储的，都是尺寸为 92×112 ，包含 256 个灰度级的灰度图像。图像文件被放置在 40 个不同的目录内，目录名用 3 位数字的序号表示，例如“010”表示第 10 个目录（文件夹）。每个目录对应一个不同的人，里面有 10 幅被拍摄对象的不同图像，并用两位数字作为文件名，例如，05.pgm 表示第 5 幅人脸图像。

在搜索引擎内输入“AT&T Facedatabase”，可以方便地找到由剑桥大学维护的这个资源。

3. Yale Facedatabase A

该数据集也被称为“Yalefaces（耶鲁人脸数据库）”。该数据库由 15 人（14 名男性、1 名女性）的人脸图像组成，每人都有 11 幅灰度图像。该数据集内的人脸图像在光线条件（中心光、左光、右光）、面部表情（高兴、正常、悲伤、困倦、惊讶、眨眼）和眼镜（戴眼镜/不戴眼镜）等方面都有变化。

4. Extended Yale Facedatabase B

扩展的耶鲁人脸数据库 B 包含 28 个人在 9 个姿势和 64 个照明条件下的 16 128 幅图像。

5. color FERET Database

FERET（Facial Recognition Technology）是由美国国防部资助的计划，该计划旨在开发用于人脸自动识别的新技术和算法。为了方便研究，FERET 计划在 1993 年至 1996 年期间收集了人脸图像，制作成数据库。该数据库用于开发、测试和评估人脸识别算法。

这个数据库包含 1564 组，共计 14 126 幅图像。所有这些图像，是由 1199 个不同的被拍摄对象及 365 组重复拍摄对象的图像构成的。其中的 365 组重复拍摄对象，是指被拍摄对象在已经完成第 1 组拍摄的情况下，在不同的时间又拍摄了第 2 组图像。

其中，部分被拍摄对象两次参与拍摄的时间间隔可能超过了两年，部分被拍摄对象多次参与拍摄。在不同时间拍摄重复的对象，使得研究人员能够首次研究人脸在经过一段时间后外观上所出现的变化。

该数据库是人脸识别领域应用最广泛的人脸数据库之一。

6. 人脸数据库整理网站

OpenCV 的官方文档（官网）推荐了在线数据集合 <http://face-rec.org/databases/>，其中列举了非常多的人脸数据库。